

Actividad [GRUPOS] PROYECTO (Parte I) BBDD

Resultados de Aprendizaje y Criterios de Evaluación aplicados

RA6	Desarrolla aplicaciones web de acceso a almacenes de datos, aplicando medidas para mantener la seguridad y la integridad de la información.
a	Se han analizado las tecnologías que permiten el acceso mediante programación a la información disponible en almacenes de datos.
b	Se han creado aplicaciones que establezcan conexiones con bases de datos.
c	Se ha recuperado información almacenada en bases de datos.
d	Se ha publicado en aplicaciones web la información recuperada.
e	Se han utilizado conjuntos de datos para almacenar la información.
f	Se han creado aplicaciones web que permitan la actualización y la eliminación de información disponible en una base de datos.
g	Se han probado y documentado las aplicaciones web.

Todos los apartados contribuyen por igual a la consecución de los CE asociados al RA.

Enunciado

En grupos, cread un proyecto, desde cero, siguiendo el patrón MVC (Modelo – Vista – Controlador) con Node JS y Express, utilizando archivo de configuración para variables de entornos ENV. En esta primera parte pondremos las bases del desarrollo de una academia (o gestor de cursos), añadiendo las conexiones a bases de datos y cumpliendo los siguientes requisitos:

- Utilizad el patrón MVC (Modelo – Vista – Controlador):
 - Directorio “**Models**” que almacenará los modelos de datos de la BBDD, en este caso:
 - **MongoDB** --> Modelo “**courses**”, que almacenará la información sobre los cursos siguiente (*podéis adaptarla a las necesidades... prácticamente todos los campos son obligatorios*):
 - `_id: ObjectId,`
 - `title: String,`
 - `description: String,`
 - `category: String, // Ej: 'Desarrollo Web', 'Marketing', etc.`
 - `startDate: Date,`
 - `endDate: Date,`
 - `location: String,`
 - `createdAt: Date,`
 - `updatedAt: Date`
 - **MongoDB** --> Modelo “**users**”, que almacenará la información sobre usuarios matriculados (alumnos, profesores, administradores...) siguiente (*podéis adaptarla a las necesidades... prácticamente todos los campos son obligatorios*):
 - `_id: ObjectId,`
 - `dni: String, // único`
 - `firstName: String,`
 - `lastName: String,`
 - `email: String,`
 - `phone: String,`
 - `birthDate: Date,`
 - `createdAt: Date,`
 - `updatedAt: Date`
 - `profile //rol o perfil de usuario, siendo ADMINISTRADOR, PROFESOR o ALUMNO`
 - **MongoDB** --> Modelo “**enrollments**”, que almacenará las matrículas de un usuario en un curso. Es decir, servirá de “puente” relacional entre el modelo “courses” y el modelo

"users". Necesitaremos los siguientes campos (podéis adaptarla a las necesidades... prácticamente todos los campos son obligatorios):

- `_id: ObjectId`,
- `courseId: ObjectId`, // referencia a Course
- `userId: ObjectId`, // referencia a User
- `enrollmentDate: Date`,
- `status: String`, // 'pendiente', 'confirmada', 'cancelada'
- `notes: String`
- **RESTRICCIÓN IMPORTANTE A TENER EN CUENTA, EN EL MODELO DE DATOS, A LA HORA DE INSERTAR MATRÍCULAS:**
 - **Un curso puede tener muchos usuarios matriculados pero un usuario sólo puede estar matriculado en un único curso a la vez (por cuestión de solapamiento de fechas), siendo, pues, una relación 1:N.**
- Directorio "**Controllers**", que almacenará los **3 controladores** permitentes (para trabajar con los 3 modelos de datos creados) con las funciones que serán ejecutadas por los Servicios, a partir de las rutas, y devolverán una respuesta (response) a partir de una petición (request). Ejemplos:
 - `course.controller`
 - `user.controller`
 - `Enrollement.controller`
- Directorio "**Services**" con la lógica de negocio que realizará toda la operativa de BBDD junto con el modelo (Models) y cuyas funciones serán consumidas por sus respectivos controladores:
 - `course.service`
 - `user.service`
 - `enrollement.service`
- Directorio "**Routes**", que almacenarán las rutas de acceso al API REST y llamarán a los controladores anteriores. Ejemplos:
 - `course.routes`
 - `user.routes`
 - `enrollement.routes`
- Directorio "**Views**" con las **vistas mínimas necesarias** para poder probar bien el API Rest al completo. Si podéis, añadid mejoras visuales CSS, accesibilidad, navegación satisfactoria, experiencia de usuario, etc.
 - Listar, Crear, Editar, Eliminar y visualizar Detalles de un Curso.
 - Listar, Crear, Editar, Eliminar y visualizar Detalles de un Usuario.
 - Listar, Crear, Editar y Eliminar matrículas de un usuario en un curso. **Utiliza POPULATE** (de MongoDB – Mongoose) con el objetivo de obtener el listado de las matrículas pero, en lugar

de mostrar los IDs de los cursos y usuarios, mostrando sus datos reales (como nombre, apellidos, nombre del curso...). Aplica esta lógica visual al resto de vistas que forman parte de las matrículas.

- Ejemplo de estructura (creando subdirectorios dentro de la carpeta "views")
 - views
 - courses
 - index.ejs
 - show.ejs
 - new.ejs
 - edit.ejs
 - users
 - index.ejs
 - show.ejs
 - new.ejs
 - edit.ejs
 - enrollements
 - index.ejs
 - new.ejs
 - edit.ejs
 - Ejemplo de renderizado:
 - `res.render("users/show.ejs")`
- Crea un archivo **.ENV** para almacenar todas aquellas variables de entornos que consideres (por ejemplo, el puerto de la aplicación y cualquier otra que consideréis). Úsalas en tu API REST.
- Documenta, mediante comentarios en código, la máxima cantidad de información posible dentro del API REST.
- Añade una carpeta de **SEEDS**, con los archivos necesarios, para crear la BBDD MongoDB con todas sus colecciones.
- Crea, levanta (en un puerto libre) y prueba, mediante las vistas creadas, que el servidor del API REST funcione correctamente. **Las operativas CRUD deben funcionar al 100%.**

Resumen (ejemplo de estructura del proyecto):

src/

```
|— models/
|   |— course.model.js
|   |— user.model.js
|   |— enrollment.model.js
```

- |— services/
 - | |— course.service.js
 - | |— user.service.js
 - | |— enrollment.service.js
- |— controllers/
 - | |— course.controller.js
 - | |— user.controller.js
 - | |— enrollment.controller.js
- |— routes/
 - | |— course.routes.js
 - | |— user.routes.js
 - | |— enrollment.routes.js
- |— **views/ (*con las vistas EJS indicadas arriba)**
 - | |— **courses/...**
 - | |— **users/...**
 - | |— **enrollments/...**
- |— seeds/ (* con los SEEDS de MongoDB – Mongoose para poder empezar a visualizar información)
- |— .env
- |— app.js

- **Entrega:**
 - Utiliza **GIT** para trabajar en grupos (tal como se ha explicado: ramas MASTER, DEVELOP y FEATURE/...). Sube tu repositorio al GIT remoto **GITHUB** y comparte la URL y Password o Token para poder hacer un "git clone". Para ello, sube un archivo TXT o PDF con dicha información a Aules.
 - Formato: **Act_Proyecto_I_Nombre_Apellidos.txt/pdf** (sólo un miembro del grupo es suficiente).
 - Además, rellena el archivo **README.md** siguiendo el modelo adjuntado en la tarea.
 - **Se penalizará el incumplimiento de estas normas de entrega.**
- **Rúbricas → Ver calificación en Aules**

***Autoevaluación y Coevaluación**

Vinculado a la tarea, existirá un cuestionario de autoevaluación y coevaluación del grupo. De esta forma, cada integrante podrá autoevaluarse explicando qué ha hecho y podrá coevaluar a sus compañeros.

Este cuestionario es totalmente confidencial y servirá para equilibrar el peso de la calificación final entre los miembros del equipo.